- .ipynb (../../../notebooks.build/strategies/gold-and-macro-factors/_sources/gold-and-macro-factors.ipynb)
- .pdf

# Gold and macro factors

## Contents

# Gold and macro factors # (#gold-and-macro-factors)

## Get packages and JPMaQS data # (#get-packages-and-jpmaqs-data)

### Set up parameters and import packages # (#set-up-parameters-and-import-packages)

```
# Import packages

import warnings

warnings.simplefilter("ignore")

import os
import numpy as np
import pandas as pd
from typing import List, Dict

import macrosynergy.management as msm
import macrosynergy.panel as msp
import macrosynergy.signal as mss
import macrosynergy.pnl as msn
import macrosynergy.visuals as msv
from macrosynergy.download import JPMaQSDownload

warnings.simplefilter("ignore")

from macrosynergy.management.types import QuantamentalDataFrame

# JPMaQS credentials
```

```
client_id: str = os.getenv("DQ_CLIENT_ID")
client_secret: str = os.getenv("DQ_CLIENT_SECRET")
if not client_id or not client_secret:
    raise ValueError("Set DQ_CLIENT_ID and DQ_CLIENT_SECRET environment variables")
start_date = "1990-01-01"

# Cross section list
cids_dm = ["USD", "EUR", "GBP", "JPY", "AUD", "CAD", "CHF", "SEK", "NOK"]

# Categories and tickers

## U.S. monetary and financial ease

cpi = ["CPIH_SA_P1M1ML12", "CPIC_SJA_P6M6ML6AR"]
ump = ["UNEMPLRATE_NSA_3MMA_D1M1ML12", "UNEMPLRATE_SA_D6M6ML6"]
crd = ["BLSCSCORE_NSA", "BLSDSCORE_NSA"]
hpi = ["HPI_SA_P1M1ML12_3MMA", "HPI_SA_P6M6ML6AR"]
ecr = ["EQCRR_NSA", "EQCRR_VT10"]
bms = ["INFTEFF_NSA"]
ease = cpi + ump + crd + hpi + ecr + bms

## Dollar stability risk indicators

gov = ["GGOBGDPRATIO_NSA", "GGSBGDPRATIO_NSA"]
rfx = ["REER_NSA_P1M12ML1", "REER_NSA_P1M60ML1"]
bvl = ["GB10YXRxEASD_NSA", "GBF10YXRxEASD_NSA"]

stabs = gov + rfx + bvl

## Global doom and crisis indicators

ccf = ["CCSCORE_SA"]
mcf = ["MBCSCORE_SA"]
wgt = ["USDGDPWGT_SA_1YMA"]

dooms = ccf + mcf + wgt

## Macroeconomic tickers

usd_macro = ["USD_" + xcat for xcat in ease + stabs]
gdm_macro = [cid + "_" + xcat for xcat in dooms for cid in cids_dm]

## Market excess return tickers

mxr = ["GLD_COXR_NSA", "GLD_COXR_VT10", "USD_EQXR_NSA", "USD_EQXR_VT10"]

# to be used after cid renaming:
rets = ["GLDXR_NSA", "GLDXR_VT10", "EQXR_NSA", "EQXR_VT10"]

## ALl tickers

tickers = usd_macro + gdm_macro + mxr
print(f"Maximum number of tickers is {len(tickers)}")
```

```
Maximum number of tickers is 48
```

```
# Download from JPMaQS via the DataQuery API

with JPMaQSDownload(
    client_id=client_id,
    client_secret=client_secret,
) as downloader:
    df = downloader.download(
        tickers=tickers,
        start_date=start_date,
        metrics=["value"],
        show_progress=False,
    )
    df = QuantamentalDataFrame(df)

print(f"Downloaded rows: {len(df):,}")
print(f"Tickers found: {len(df.list_tickers()):,d} out of {len(tickers):,d}")
```

```
Downloading data from JPMaQS.
Timestamp UTC:  2026-09-02 09:26:17
Connection successful!
Downloaded rows: 416,571
Tickers found: 48 out of 48
```

```
dfx = df.copy()
dfx.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 416571 entries, 0 to 416570
Data columns (total 4 columns):
 #   Column     Non-Null Count   Dtype
---  ------     --------------   -----
 0   real_date  416571 non-null  datetime64[ns]
 1   cid        416571 non-null  category
 2   xcat       416571 non-null  category
 3   value      416571 non-null  float64
dtypes: category(2), datetime64[ns](1), float64(1)
memory usage: 7.2 MB
```

## Availability and preprocessing # (#availability-and-preprocessing)

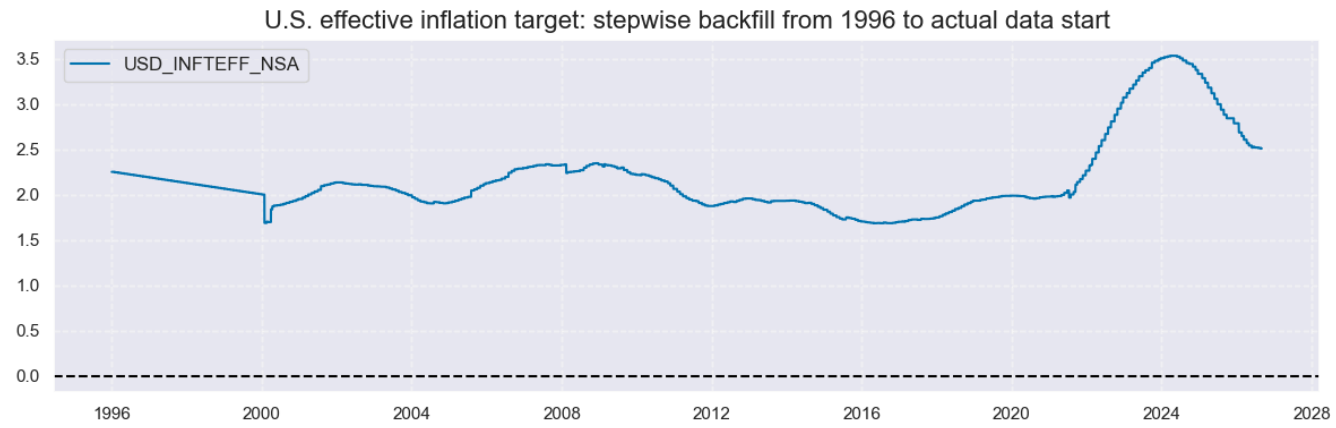
### Renamings # (#renamings)

```
# Cross-section renamings
```

```
dict_repl = {
    "GLD": "USD",
}

for key, value in dict_repl.items():
    dfx["cid"] = dfx["cid"].str.replace(key, value)

# Category name replacements

dict_repl = {
    "COXR_NSA": "GLDXR_NSA",
    "COXR_VT10": "GLDXR_VT10",
}

for key, value in dict_repl.items():
    dfx["xcat"] = dfx["xcat"].str.replace(key, value)
```

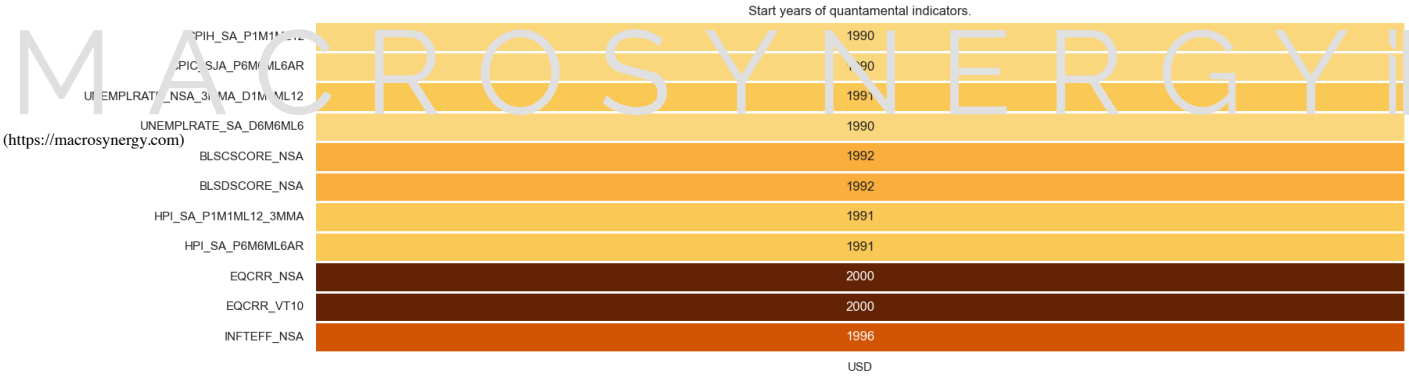
```
# Backfill U.S. effective inflation target from 1996 to the start of actual data (stepwise, monthly)

cidx = ["USD"]
xcatx = "INFTEFF_NSA"

first_date = dfx.loc[
    dfx["cid"].isin(cidx) & (dfx["xcat"] == xcatx), "real_date"
].min()

backfill_dates = pd.bdate_range(start="1996-01-01", end=first_date - pd.Timedelta(days=1))

step_dates = pd.date_range(start="1996-01-01", end="2000-01-01", freq="MS")
step_values = np.linspace(2.25, 2, len(step_dates))
backfill_values = (
    pd.Series(step_values, index=step_dates).reindex(backfill_dates, method="ffill").values
)

# Alternative: smooth linear daily backfill instead of stepwise
# backfill_values = np.linspace(2.25, 2, len(backfill_dates))

dfb = pd.DataFrame(
    {
        "real_date": backfill_dates,
        "cid": cidx[0],
        "xcat": xcatx,
        "value": backfill_values,
    }
)

dfx = msm.update_df(dfx, dfb)
```

```
# Check backfilled inflation target

cidx = ["USD"]
xcatx = ["INFTEFF_NSA"]
start = "1996-01-01"

msp.view_timelines(
    df=dfx,
    xcats=xcatx,
    cids=cidx,
    title="U.S. effective inflation target: stepwise backfill from 1996 to actual data start",
    title_fontsize=16,
    start=start,
    size=(12, 4)
)
```

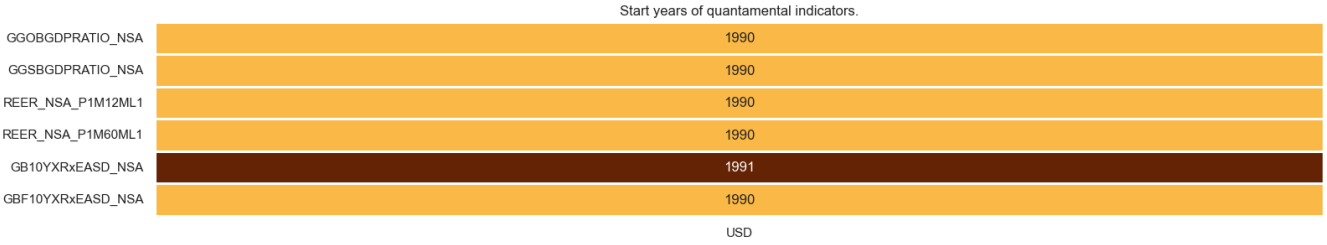



### Availability check # (#availability-check)

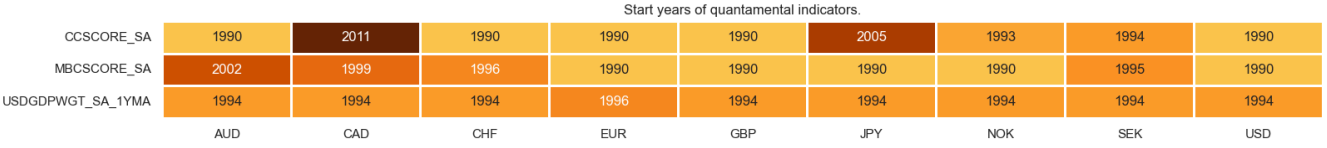
```
# Available financial ease categories

xcatx = ease
cidx = ["USD"]
msm.check_availability(dfx, xcats=xcatx, cids=cidx, missing_recent=False)
```




```
# Available stability concern categories

xcatx = stabs
cidx = ["USD"]
msm.check_availability(dfx, xcats=xcatx, cids=cidx, missing_recent=False)
```




```
# Available doom and crisis categories

xcatx = dooms
cidx = cids_dm
msm.check_availability(dfx, xcats=xcatx, cids=cidx, missing_recent=False)
```



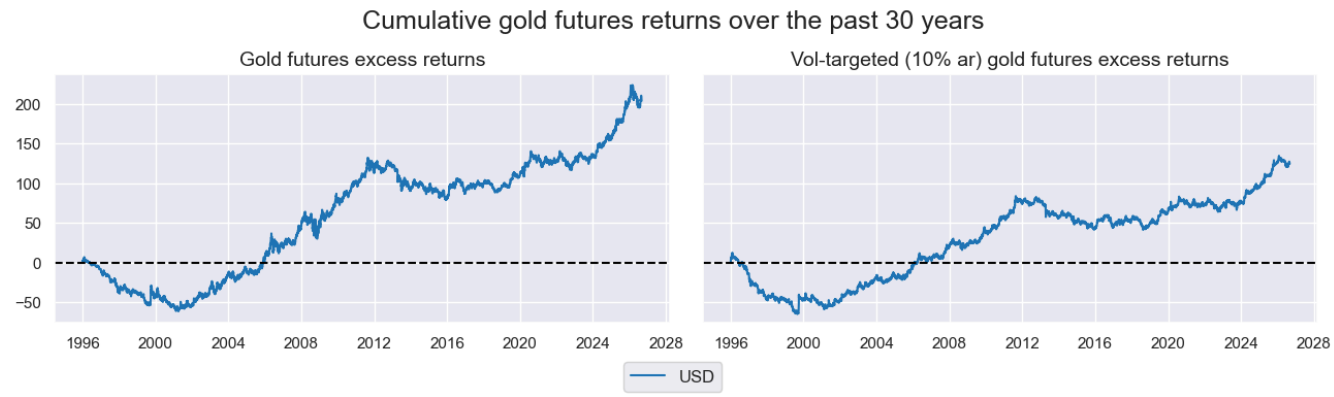

```
# Available market excess return categories

xcatx = rets
cidx = ["USD"]
msm.check_availability(dfx, xcats=xcatx, cids=cidx, missing_recent=False)
```


## Targets # (#targets)

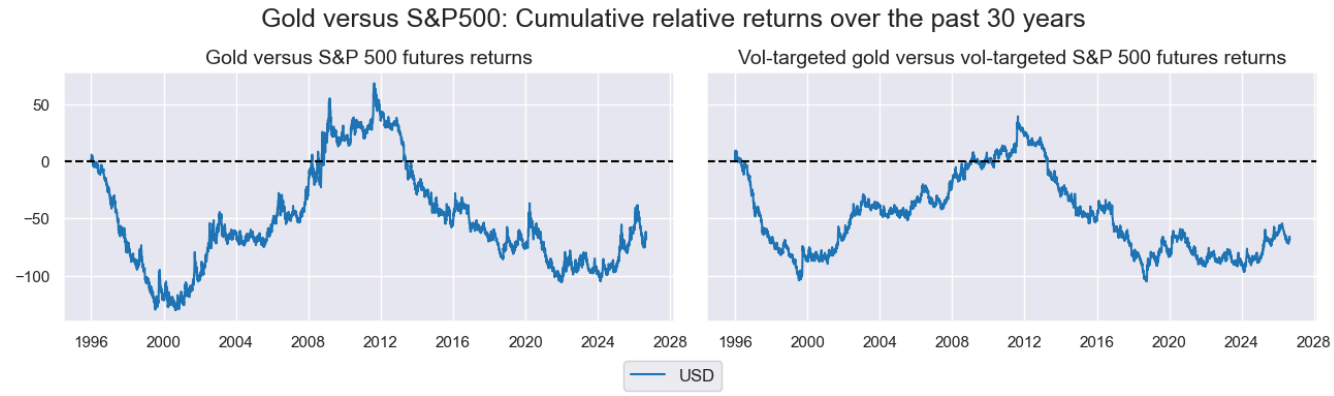
```
# Gold vs. S&P 500 excess return

cidx = ["USD"]

calcs = [
    "GLDvEQXR_NSA = GLDXR_NSA - EQXR_NSA",
    "GLDvEQXR_VT10 = GLDXR_VT10 - EQXR_VT10",
]
dfa = msp.panel_calculator(dfx, calcs=calcs, cids=cidx)

dfx = msm.update_df(dfx, dfa)

grets = ["GLDXR_NSA", "GLDXR_VT10", "GLDvEQXR_NSA", "GLDvEQXR_VT10"]

labels_grets = {
    "GLDXR_NSA": "Gold futures excess returns",
    "GLDXR_VT10": "Vol-targeted (10% ar) gold futures excess returns",
    "GLDvEQXR_NSA": "Gold versus S&P 500 futures returns",
    "GLDvEQXR_VT10": "Vol-targeted gold versus vol-targeted S&P 500 futures returns"
}

# Check gold futures returns

cidx = ["USD"]
xcatx = ["GLDXR_NSA", "GLDXR_VT10"]
start = "1996-01-01"
```

```
msp.view_timelines(
    df=dfx,
    xcats=xcatx,
    cids=cidx,
    xcat_grid=True,
    same_y=True,
    title="Cumulative gold futures returns over the past 30 years",
    title_fontsize=18,
    ncol=2,
    height=2.5,
    aspect=1.75,
    cumsum=True,
    xcat_labels=labels_grets,
    start=start
)
```






```
# Check gold futures returns

cidx = ["USD"]
xcatx = ["GLDvEQXR_NSA", "GLDvEQXR_VT10"]
start = "1996-01-01"

msp.view_timelines(
    df=dfx,
    xcats=xcatx,
    cids=cidx,
    xcat_grid=True,
    same_y=True,
    title="Gold versus S&P500: Cumulative relative returns over the past 30 years",
    title_fontsize=18,
    ncol=2,
    height=2.5,
    aspect=1.75,
    cumsum=True,
    xcat_labels=labels_grets,
    start=start
)
```

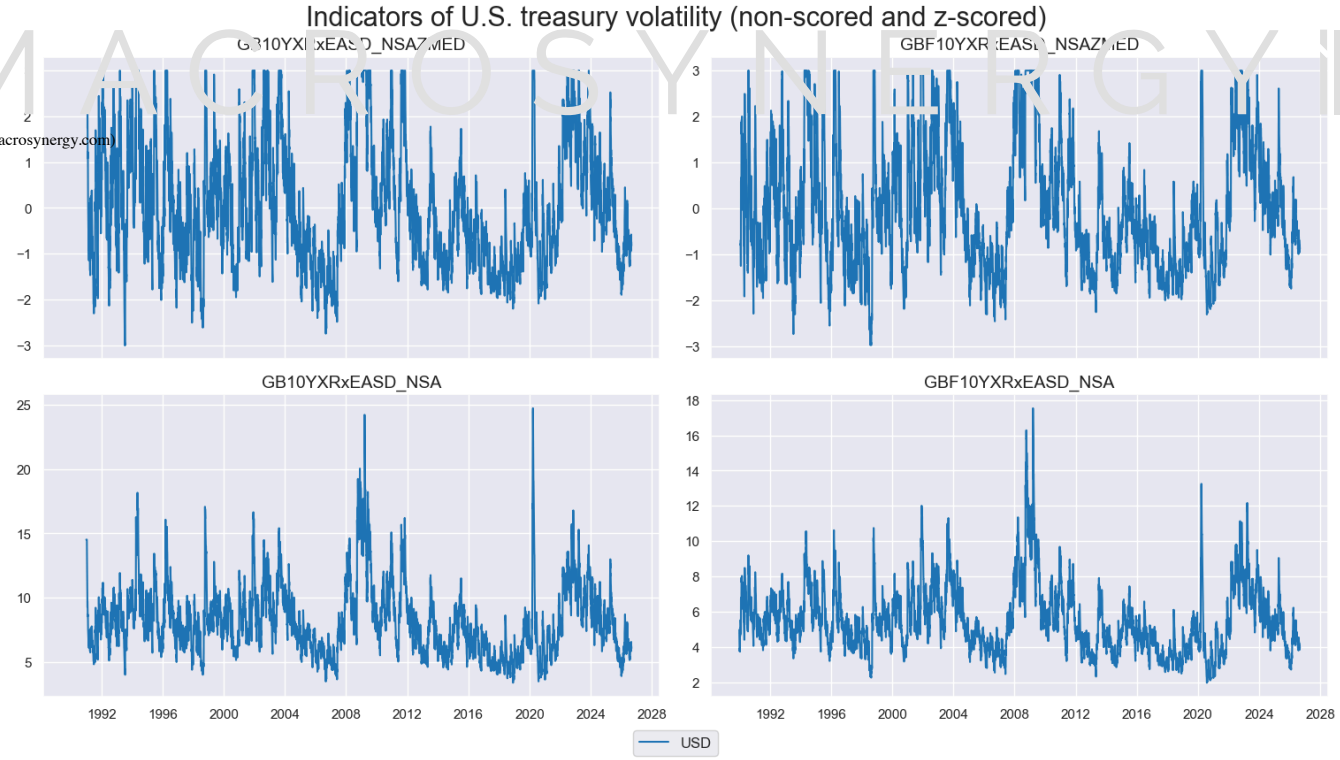


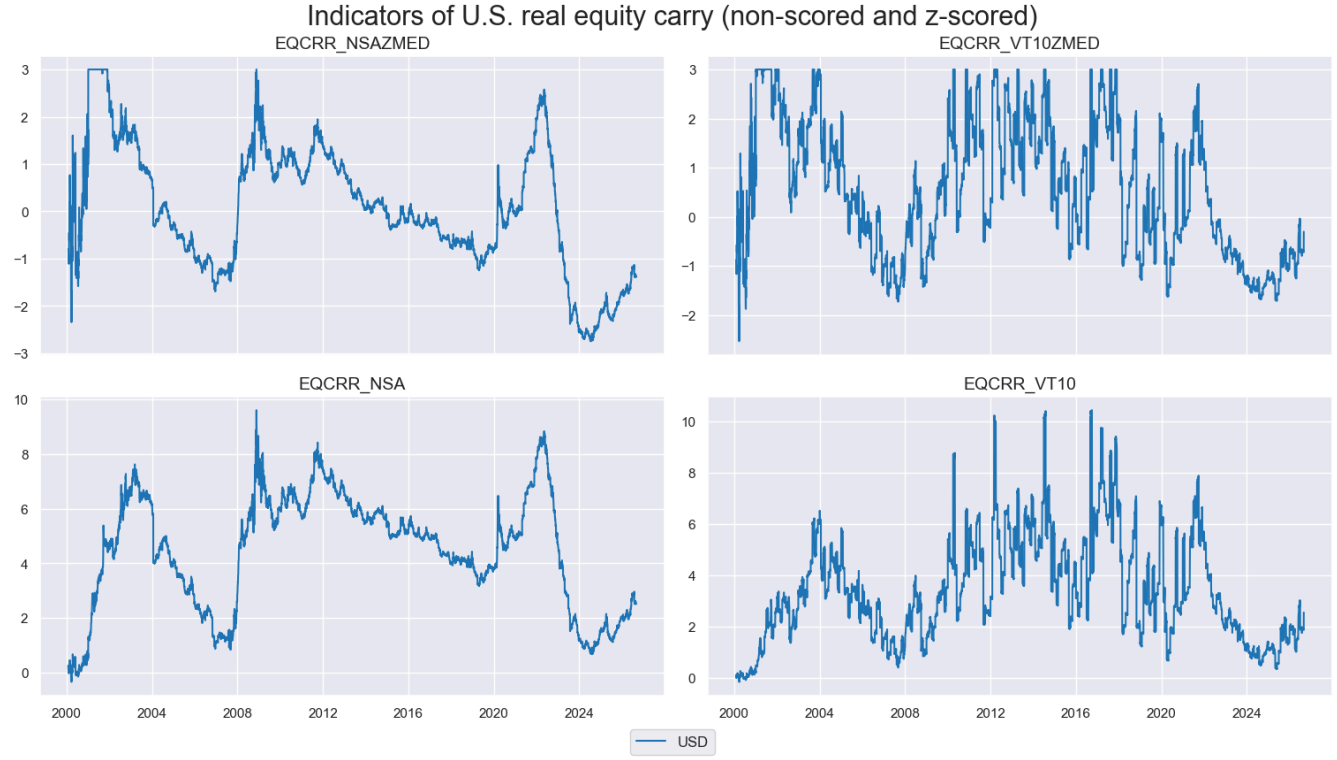

## Features # (#features)

### Preparatory category transformations # (#preparatory-category-transformations)

```
# Excess inflation rates

cidx = ["USD"]
infs = cpi + hpi

calcs = [f"X{inf} = {inf} - INFTEFF_NSA" for inf in infs]
dfa = msp.panel_calculator(dfx, calcs=calcs, cids=cidx)

dfx = msm.update_df(dfx, dfa)

xhpi = [f"X{inf}" for inf in hpi]
xcpi = [f"X{inf}" for inf in cpi]

# Excess government balance (adding 3% deficit allowance)
```

```
cidx = ["USD"]

calcs = [f"X{gov} = {gov} + 3" for gov in gov]
df = msp.panel_calculator(dfx, calcs=calcs, cids=cidx)

dfx = msm.update_df(dfx, df)
xgov = [f"X{gov}" for gov in gov]

# Excess U.S. bond volatility and equity carry scores

cidx = ["USD"]
xcatx = bvl + ecr

dfa = msp.make_zn_scores(
    dfx,
    xcat=xcatx,
    cids=cidx,
    neutral="median",
    thresh=3,
    est_freq="M",
    postfix="ZMED",
)

dfx = msm.update_df(dfx, dfa)

xbvl = [f"{xcat}ZMED" for xcat in bvl]
xecr = [f"{xcat}ZMED" for xcat in ecr]

# Check scoring

cidx = ["USD"]
xcatx = xbvl + bvl

msv.timelines(
    df=dfx,
    xcats=xcatx,
    cids=cidx,
    title="Indicators of U.S. treasury volatility (non-scored and z-scored)",
    xcat_grid=True,
    same_y=False,
    ncol=2,
)

xcatx = xecr + ecr

msv.timelines(
    df=dfx,
    xcats=xcatx,
    cids=cidx,
    xcat_grid=True,
    title="Indicators of U.S. real equity carry (non-scored and z-scored)",
    same_y=False,
    ncol=2,
)
```

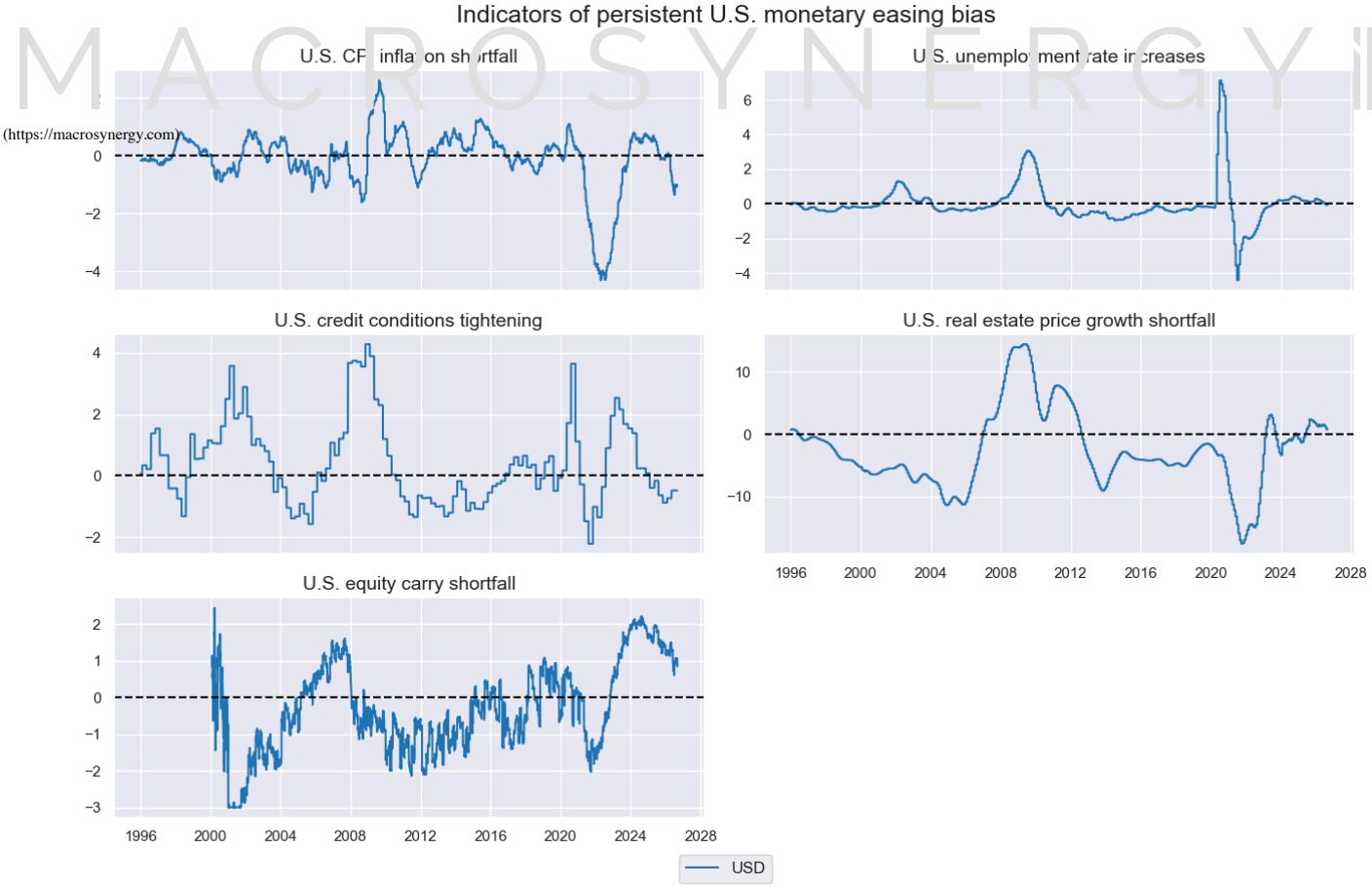


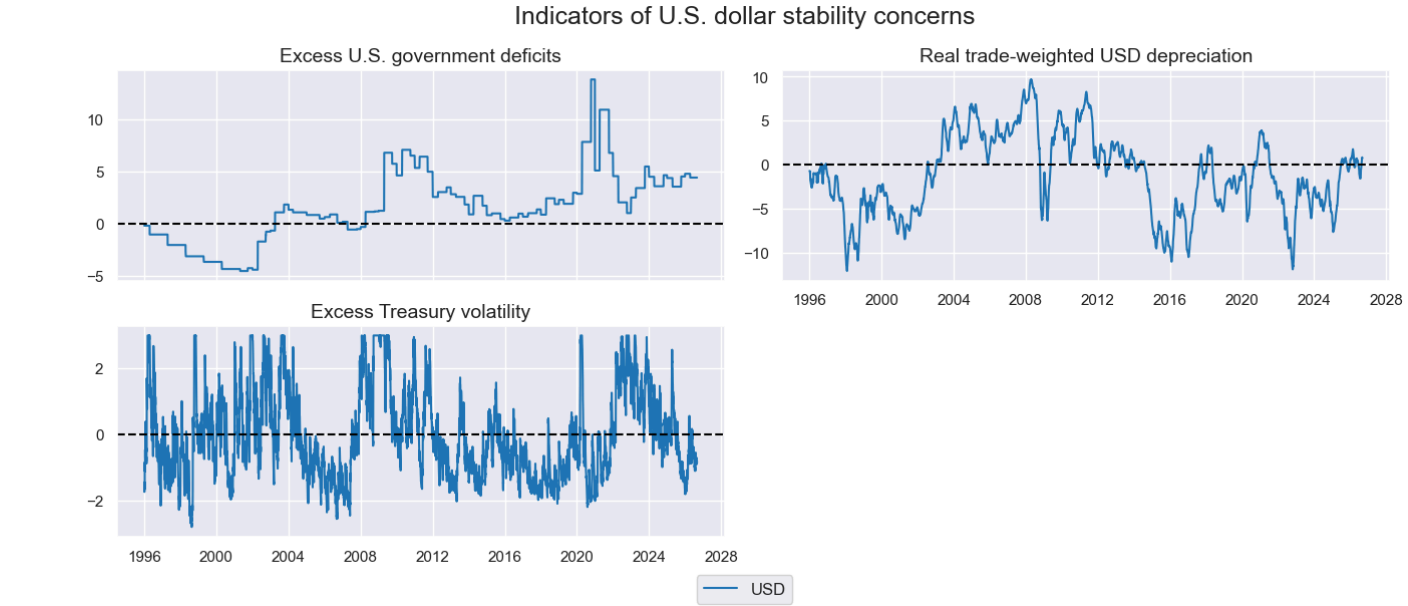




**Conceptual factor generation # (#conceptual-factor-generation)**

```
# Conceptual factor dictionaries

dict_ease = {
    "CPISF": {"comps": xcpi, "sign": "neg"},
    "UMPRISE": {"comps": ump, "sign": "pos"},
    "LOANTIGHT": {"comps": crd, "sign": "neg"},
    "HPISF": {"comps": xhpi, "sign": "neg"},
    "ECRSF": {"comps": xecr, "sign": "neg"},
}

dict_stabs = {
    "XGOVDEF": {"comps": xgov, "sign": "neg"},
    "DEVAL": {"comps": rfx, "sign": "neg"},
    "XVOL": {"comps": xbvl, "sign": "pos"},
}

dict_doom = {
    "GLBCCDOOM": {"comps": ["CCSCORE_SA"], "sign": "neg"},
```

```
    "GLBMCDOOM": {"comps": ["MBCSCORE_SA"], "sign": "neg"},
}

# Factor lists

ease_factors = list(dict_ease.keys())
stab_factors = list(dict_stabs.keys())
doom_factors = list(dict_doom.keys())

# Labelling conceptual factors

labels_concepts = {
    "CPISF": "U.S. CPI inflation shortfall",
    "UMPRISE": "U.S. unemployment rate increases",
    "LOANTIGHT": "U.S. credit conditions tightening",
    "HPISF": "U.S. real estate price growth shortfall",
    "ECRSF": "U.S. equity carry shortfall",
    "XGOVDEF": "Excess U.S. government deficits",
    "DEVAL": "Real trade-weighted USD depreciation",
    "XVOL": "Excess Treasury volatility",
    "GLBCCDOOM": "Global consumer doom score",
    "GLBMCDOOM": "Global industrial doom score",
}

# U.S. factors calculations

cidx = ["USD"]
dict_us = dict_ease | dict_stabs

dfa = pd.DataFrame(columns=dfx.columns)

for key, value in dict_us.items():
    xcatx = value["comps"]
    sign = value["sign"]
    dfaa = msp.linear_composite(
        df=dfx,
        xcats=xcatx,
        cids=cidx,
        new_xcat=key,
    )
    if sign == "neg":
        dfaa = msp.panel_calculator(
            df=dfaa,
            calcs=[f"{key} = -1 * {key}"],
            cids=cidx,
        )
    dfa = msm.update_df(dfa, dfaa)

dfx = msm.update_df(dfx, dfa, xcat_replace=True)

# Check U.S. factors

cidx = ["USD"]
xcatx = ease_factors
start = "1996-01-01"

msp.view_timelines(
    df=dfx,
    xcats=xcatx,
    cids=cidx,
    xcat_grid=True,
    same_y=False,
    title="Indicators of persistent U.S. monetary easing bias",
    title_fontsize=18,
    ncol=2,
    height=2,
    aspect=2.25,
    xcat_labels=labels_concepts,
    start=start,
)

xcatx = stab_factors
msp.view_timelines(
    df=dfx,
    xcats=xcatx,
    cids=cidx,
    xcat_grid=True,
    same_y=False,
    title="Indicators of U.S. dollar stability concerns",
    title_fontsize=18,
    ncol=2,
    height=2,
    aspect=2.25,
    xcat_labels=labels_concepts,
    start=start,
)
```



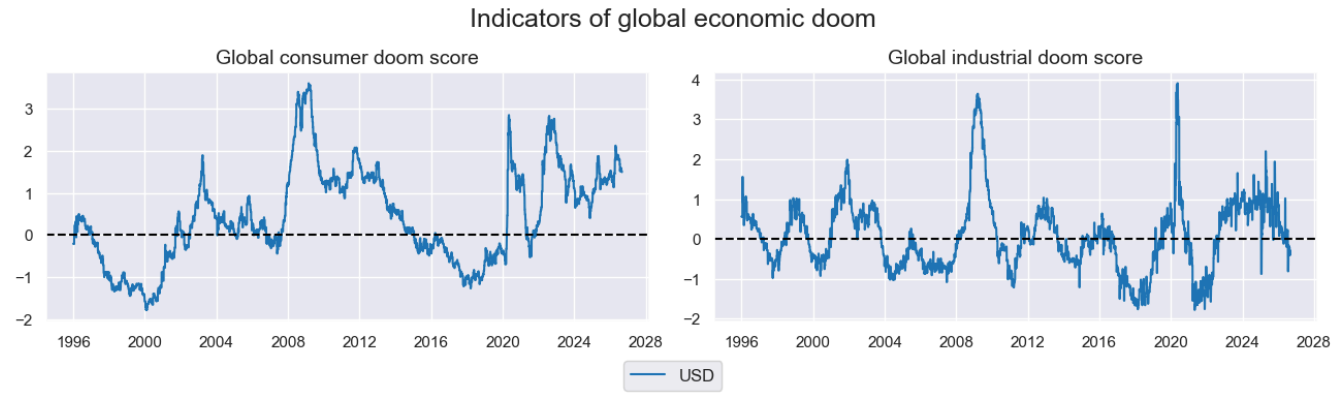

```
# Global doom factors

cidx = cids_dm

dfa = pd.DataFrame(columns=dfx.columns)

for key, value in dict_doom.items():
    xcat = value["comps"]
    sign = value["sign"]
    dfaa = msp.linear_composite(
        df=dfx,
        xcats=xcat,
        cids=cidx,
        new_cid="USD",
        weights="USDGDPWGT_SA_1YMA"
    )
    dfaa['xcat'] = key
    if sign == "neg":
        dfaa = msp.panel_calculator(
            df=dfaa,
```

```
            calcs=[f"{key} = -1 * {key}"],
            cids=["USD"],
        )

    dfa = msm.update_df(dfa, dfaa)

dfx = msm.update_df(dfx, dfa, xcat_replace=True)
```



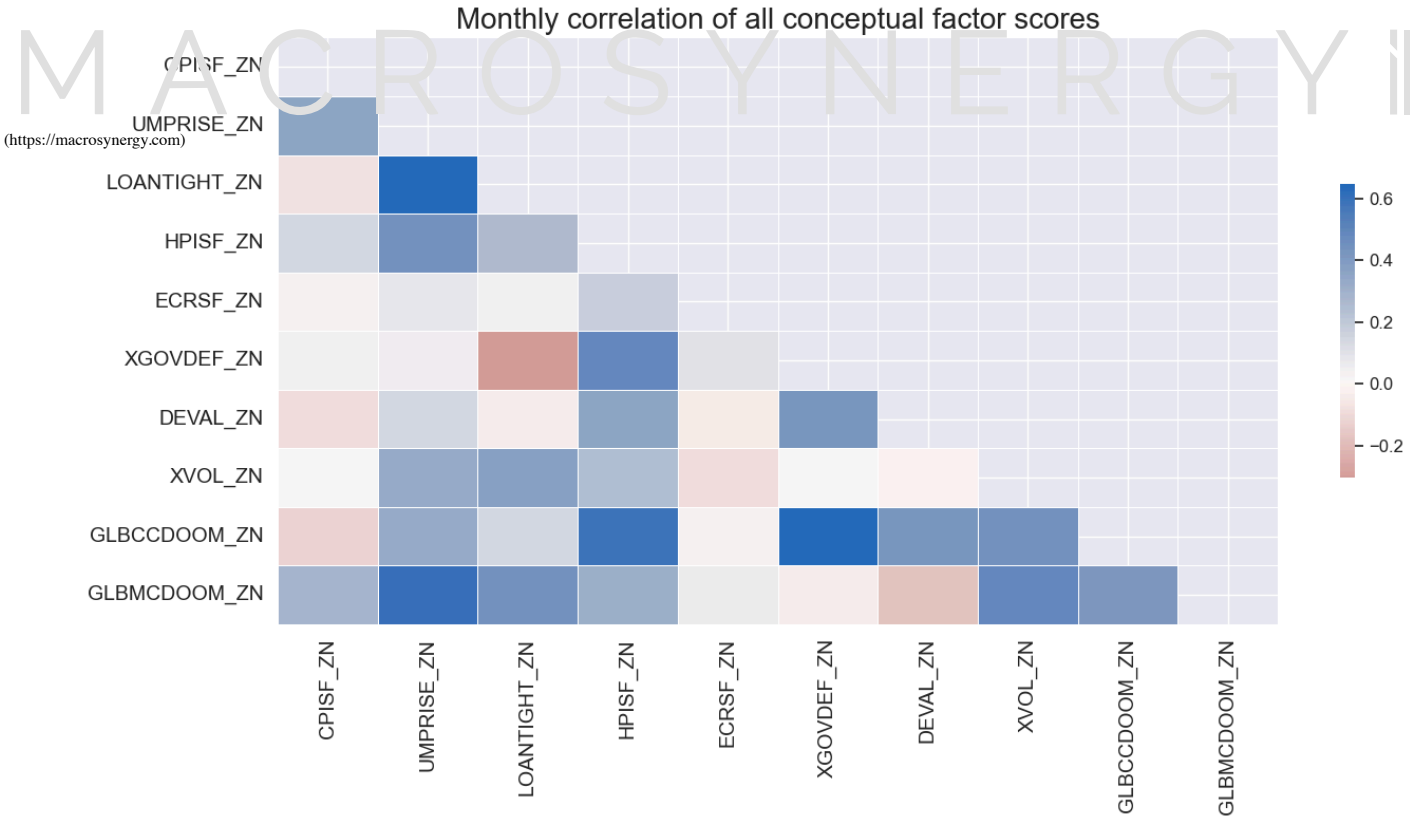
(https://macrosynergy.com)

```
# Check global doom factors

cidx = ["USD"]
xcatx = doom_factors

msp.view_timelines(
    df=dfx,
    xcats=xcatx,
    cids=cidx,
    xcat_grid=True,
    same_y=False,
    title="Indicators of global economic doom",
    title_fontsize=18,
    ncol=2,
    height=2.5,
    aspect=1.75,
    xcat_labels=labels_concepts,
    start=start,
)
```




### Conceptual factor scores # (#conceptual-factor-scores)

```
# Score all conceptual factors by the same method

cidx = ["USD"]
xcatx = ease_factors + stab_factors + doom_factors

dfa = msp.make_zn_scores(
    df=dfx,
    xcat=xcatx,
    cids=cidx,
    neutral="zero",
    thresh=3,
    est_freq="M",
    postfix="_ZN",
    pan_weight=0
)

dfx = msm.update_df(dfx, dfa, xcat_replace=True)

ease_factors_zn = [f"{xcat}_ZN" for xcat in ease_factors]
stab_factors_zn = [f"{xcat}_ZN" for xcat in stab_factors]
doom_factors_zn = [f"{xcat}_ZN" for xcat in doom_factors]

# Correlation of all conceptual factor scores

xcatx = ease_factors_zn + stab_factors_zn + doom_factors_zn
cidx = ["USD"]

msp.correl_matrix(
    df=dfx,
    xcats=xcatx,
    cids=cidx,
    freq="M",
    start=start,
    title="Monthly correlation of all conceptual factor scores",
    title_fontsize=20,
)
```



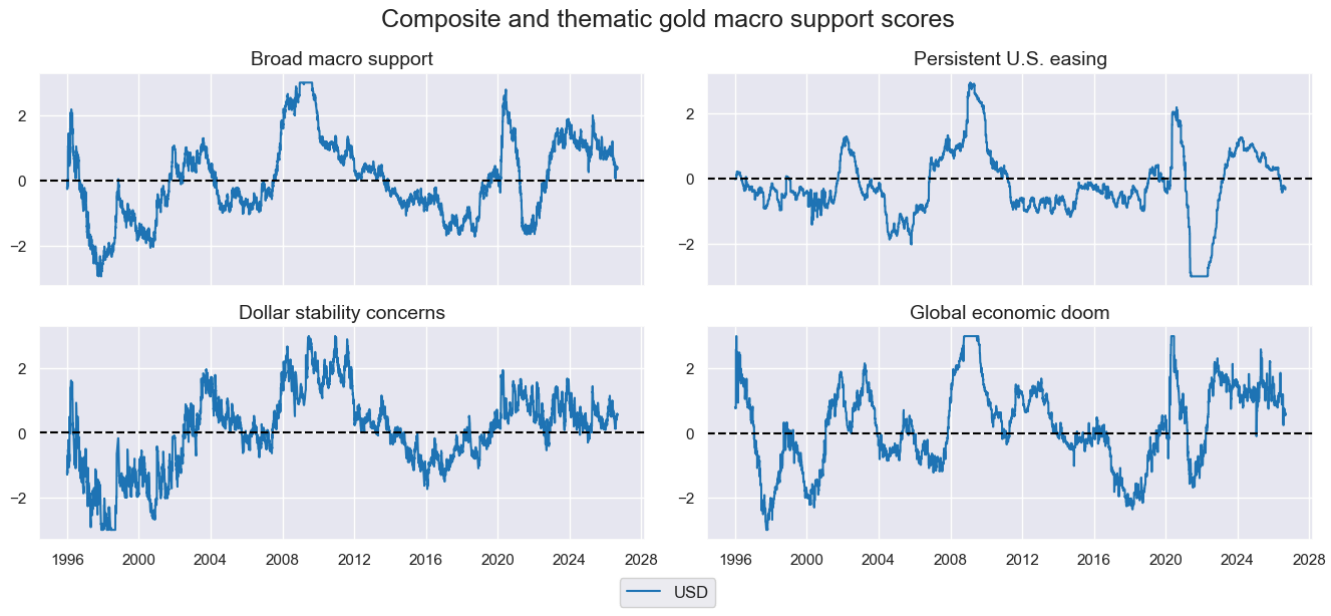

### Thematic factor scores # (#thematic-factor-scores)

```
# Thematic factor dictionary

dict_themes = {
    "FINMON_EASE": {"comps": ease_factors_zn, "label": "Financial and monetary ease"},
    "DOLLAR_STAB": {"comps": stab_factors_zn, "label": "Dollar stability risk"},
    "GLOBAL_DOOM": {"comps": doom_factors_zn, "label": "Global economic doom"},
}

theme_factors = list(dict_themes.keys())

# Thematic factor calculations

# U.S. factors calculations

cidx = ["USD"]
dict_us = dict_themes

dfa = pd.DataFrame(columns=dfx.columns)

for key, value in dict_us.items():
    xcatx = value["comps"]
    dfaa = msp.linear_composite(
        df=dfx,
        xcats=xcatx,
        cids=cidx,
        new_xcat=key,
    )
    dfa = msm.update_df(dfa, dfaa)

dfx = msm.update_df(dfx, dfa, xcat_replace=True)

# Re-scoring thematic factors

cidx = ["USD"]
xcatx = theme_factors

dfa = msp.make_zn_scores(
    df=dfx,
    xcat=xcatx,
    cids=cidx,
    neutral="zero",
    thresh=3,
    est_freq="M",
    postfix="_ZN",
    pan_weight=0
)

dfx = msm.update_df(dfx, dfa, xcat_replace=True)

theme_factors_zn = [f"{xcat}_ZN" for xcat in theme_factors]
```

### Composite macro support score # (#composite-macro-support-score)

```
# Composite gold macro support score
```

```
cidx = ["USD"]
xcatx = theme_factors_zn

dfa = msp.linear_composite(
    df=dfx,
    xcats=xcatx,
    cids=cidx,
    new_xcat="MACRO_SUPPORT",
)

dfx = msm.update_df(dfx, dfa, xcat_replace=True)

xcatx = ["MACRO_SUPPORT"]
dfa = msp.make_zn_scores(
    df=dfx,
    xcat=xcatx,
    cids=cidx,
    neutral="zero",
    thresh=3,
    est_freq="M",
    postfix="_ZN",
    pan_weight=0
)

dfx = msm.update_df(dfx, dfa, xcat_replace=True)

macro_factors_zn = ["MACRO_SUPPORT_ZN"] + theme_factors_zn

# Labelling thematic factors

labels_themes_scores = {
    "MACRO_SUPPORT_ZN": "Broad macro support",
    "FINMON_EASE_ZN": "Persistent U.S. easing",
    "DOLLAR_STAB_ZN": "Dollar stability concerns",
    "GLOBAL_DOOM_ZN": "Global economic doom",
}

# Check macro support factor scores

cidx = ["USD"]
xcatx = macro_factors_zn
start = "1996-01-01"

msp.view_timelines(
    df=dfx,
    xcats=xcatx,
    cids=cidx,
    xcat_grid=True,
    same_y=False,
    title="Composite and thematic gold macro support scores",
    title_fontsize=18,
    ncol=2,
    height=2,
    aspect=2.25,
    xcat_labels=labels_themes_scores,
    start=start,
)
```

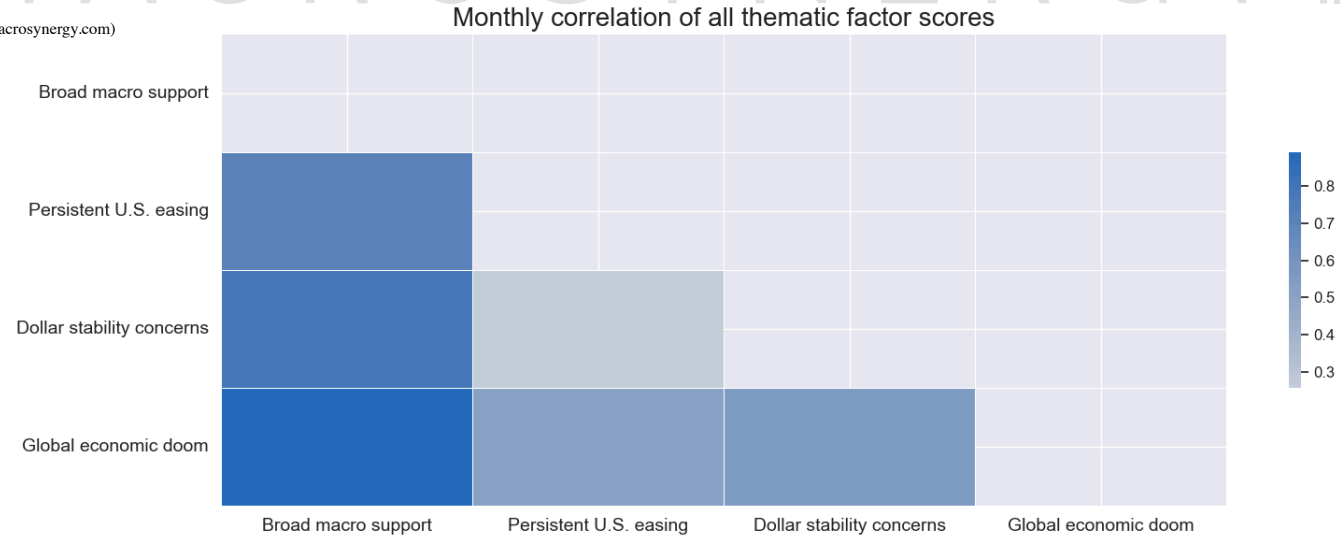



```
# Monthly correlation of all thematic factor scores

xcatx = macro_factors_zn
cidx = ["USD"]

msp.correl_matrix(
    df=dfx,
    xcats=xcatx,
    cids=cidx,
    freq="M",
    start=start,
```

```
    title="Monthly correlation of all thematic factor scores",
    title_fontsize=20,
    size=(6, 6),
    cat_labels=labels_themes_scores,
)
```



(https://macrosynergy.com)



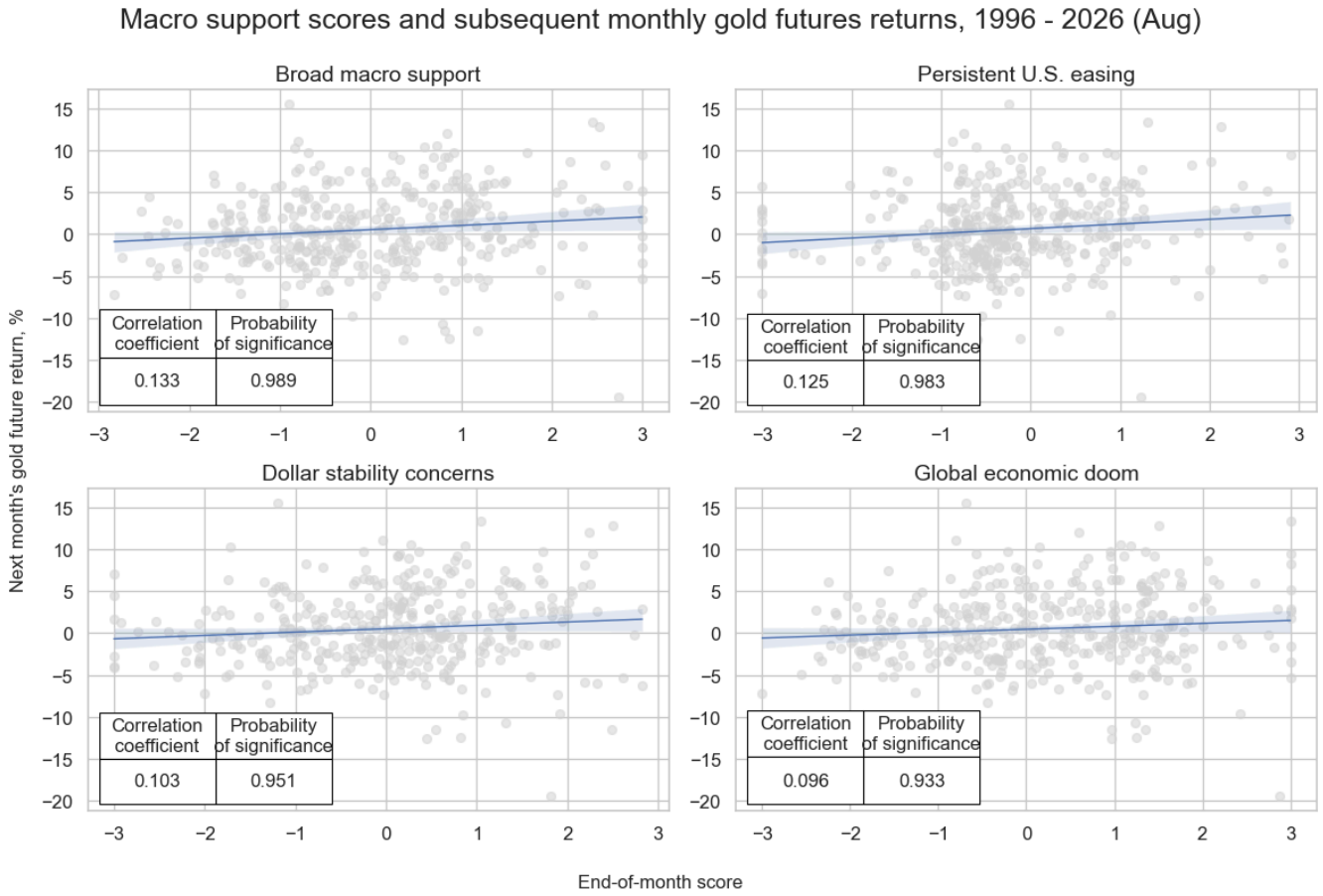

## Value check # (#value-check)

### Gold outright # (#gold-outright)

#### Specs and regression test # (#specs-and-regression-test)

```
# Directional specs

dict_gd = {
    "sigx": macro_factors_zn,
    "targx": ['GLDXR_NSA', 'GLDXR_VT10'],
    "cidx": ["USD"],
    "start": "1996-01-01",
    "crs": None,
    "srr": None,
    "pnls": None,
}

# Directional test regressions (monthly)

dix = dict_gd

sigs = dix["sigx"]
targs = dix["targx"]
cidx = dix["cidx"]
start = dix["start"]

# Initialize the dictionary to store CategoryRelations instances

dict_cr = {}

for targ in targs[:1]:
    for sig in sigs:
        lab = sig + "_" + targ
        dict_cr[lab] = msp.CategoryRelations(
            dfx,
            xcats=[sig, targ],
            cids=cidx,
            freq="M",
            lag=1,
            xcat_aggs=["last", "sum"],
            start=start,
        )

dix["crs"] = dict_cr

# Directional plots

dix = dict_gd
dict_cr = dix["crs"]
sigs = dix["sigx"]
targs = dix["targx"]

crs = list(dict_cr.values())
crs_keys = list(dict_cr.keys())
ncol = 2
nrow = 2

scatter_labels = {
    'MACRO_SUPPORT_ZN_GLDXR_NSA': "Broad macro support",
    'FINMON_EASE_ZN_GLDXR_NSA': "Persistent U.S. easing",
    'DOLLAR_STAB_ZN_GLDXR_NSA': "Dollar stability concerns",
    'GLOBAL_DOOM_ZN_GLDXR_NSA': "Global economic doom",
}
```

```
msv.multiple_reg_scatter(
    cat_rels=crs,
    ncol=ncol,
    nrow=nrow,
    figsize=(12, ),
    prob_est="pool",
    coef_box="lower left",
    title="Macro support scores and subsequent monthly gold futures returns, 1996 - 2026 (Aug)",
    title_fontsize=18,
    subplot_titles=[scatter_labels[cr] for cr in crs_keys],
    share_axes=False,
    xlab="End-of-month score",
    ylab="Next month's gold future return, %",
    coef_box_font_size=12,
)
```




```
# Directional test regressions (quarterly)

dix = dict_gd

sigs = dix["sigx"]
targs = dix["targx"]
cidx = dix["cidx"]
start = dix["start"]

# Initialize the dictionary to store CategoryRelations instances

dict_cr = {}

for targ in targs[:1]:
    for sig in sigs:
        lab = sig + "_" + targ
        dict_cr[lab] = msp.CategoryRelations(
            dfx,
            xcats=[sig, targ],
            cids=cidx,
            freq="Q",
            lag=1,
            xcat_aggs=["last", "sum"],
            start=start,
        )

dix["crq"] = dict_cr

# Directional plots

dix = dict_gd
dict_cr = dix["crq"]
sigs = dix["sigx"]
targs = dix["targx"]

crs = list(dict_cr.values())
```

```
crs_keys = list(dict_cr.keys())
ncol = 2
nrow = 2

scatter_labels = {
    'MACRO_SUPPORT_ZN_GLDXR_NSA': "Broad macro support",
    'FINMON_EASE_ZN_GLDXR_NSA': "Persistent U.S. easing",
    'DOLLAR_STAB_ZN_GLDXR_NSA': "Dollar stability concerns",
    'GLOBAL_DOOM_ZN_GLDXR_NSA': "Global economic doom",
}

msv.multiple_reg_scatter(
    cat_rels=crs,
    ncol=ncol,
    nrow=nrow,
    figsize=(12, 8),
    prob_est="pool",
    coef_box="lower left",
    title="Macro support scores and subsequent quarterly gold futures returns, 1996 – 2026 (Aug)",
    title_fontsize=18,
    subplot_titles=[scatter_labels[cr] for cr in crs_keys],
    share_axes=False,
    xlab="End-of-month score",
    ylab="Next month's gold future return, %",
    coef_box_font_size=12,
    # separator=2010,
)
```

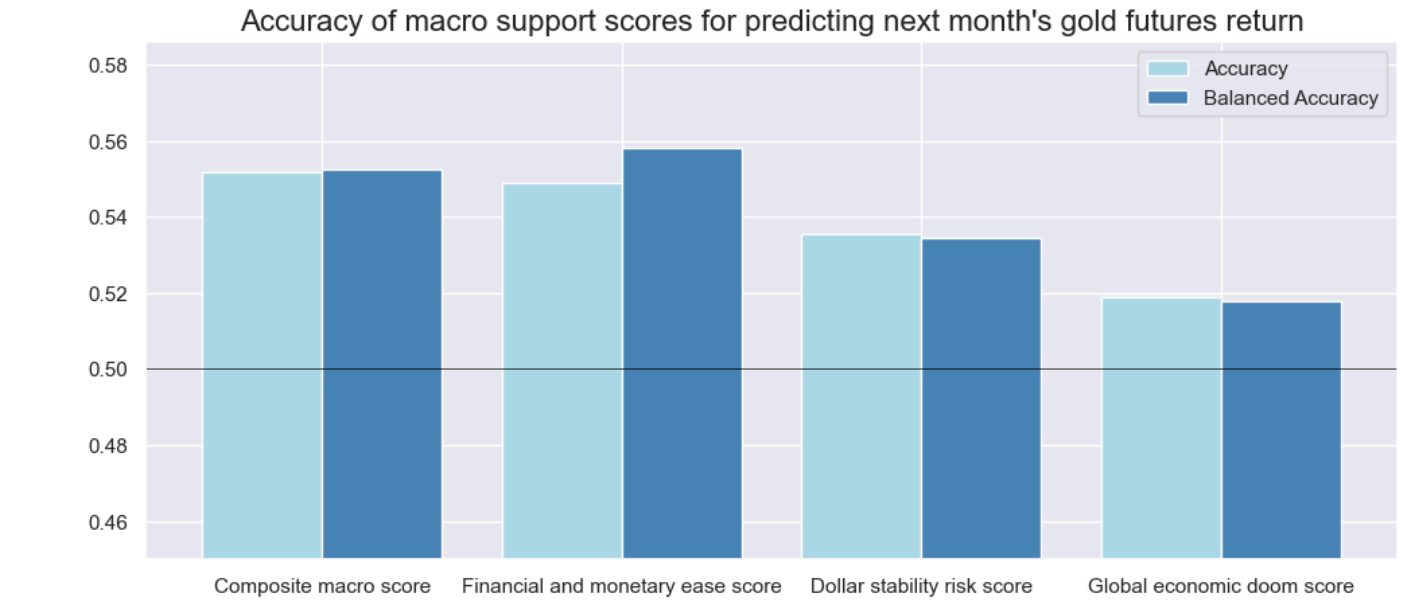

**Accuracy and correlation check # (#accuracy-and-correlation-check)**

```
# Directional accuracy

dix = dict_gd

sigs = dix["sigx"]
targs = dix["targx"][:1]
cidx = dix["cidx"]
start = dix["start"]

srr = mss.SignalReturnRelations(
    dfx,
    cids=cidx,
    sigs=sigs,
    rets=targs,
    freqs="M",
    start=start,
)

display(srr.multiple_relations_table().round(3))

dix["srr"] = srr
```

| Return | Signal | Frequency | Aggregation | accuracy | bal_accuracy | pos_sigr | pos_retr | pos_prec | neg_prec | pearson | pearson_pval | kendall | kendall_pval | auc |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| GLDXR_NSA | DOLLAR_STAB_ZN | M | last | 0.535 | 0.534 | 0.554 | 0.514 | 0.544 | 0.524 | 0.125 | 0.049 | 0.097 | 0.006 | 0.524 |
| | FINMON_EASE_ZN | M | last | 0.549 | 0.558 | 0.351 | 0.514 | 0.589 | 0.527 | 0.125 | 0.017 | 0.103 | 0.003 | 0.553 |
| | GLOBAL_DOOM_ZN | M | last | 0.519 | 0.518 | 0.554 | 0.514 | 0.529 | 0.506 | 0.096 | 0.067 | 0.077 | 0.027 | 0.518 |
| | MACRO_SUPPORT_ZN | M | last | 0.552 | 0.552 | 0.478 | 0.514 | 0.568 | 0.536 | 0.133 | 0.011 | 0.113 | 0.001 | 0.552 |



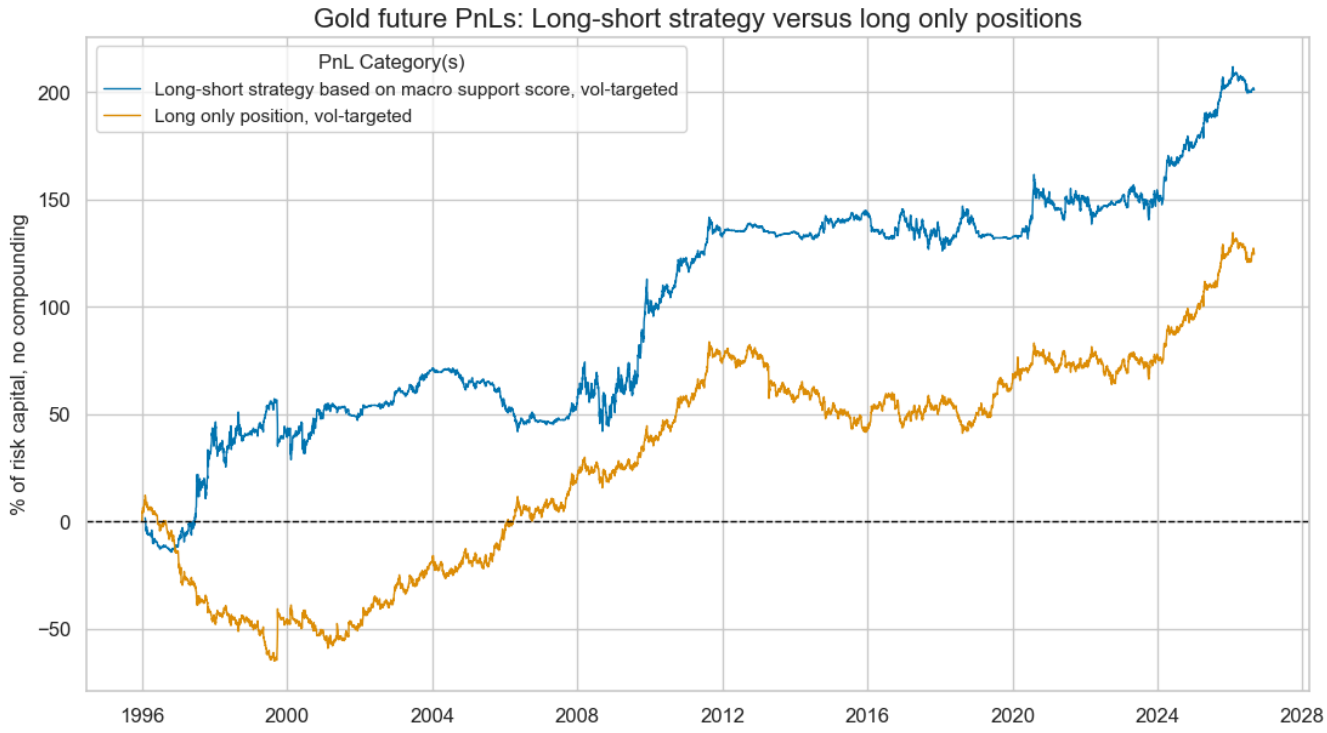
```
# Directional accuracy plots

dix = dict_gd
srr = dix["srr"]
targ = dix["targx"][0]

sig_labels = {
    'MACRO_SUPPORT_ZN': "Composite macro score",
    'FINMON_EASE_ZN': "Financial and monetary ease score",
    'DOLLAR_STAB_ZN': "Dollar stability risk score",
    'GLOBAL_DOOM_ZN': "Global economic doom score",
}

srr.accuracy_bars(
    view="signals",
    ret=targ,
    size=(12, 5),
    title="Accuracy of macro support scores for predicting next month's gold futures return",
    title_fontsize=16,
    x_labels=sig_labels,
    x_label_rotation=45,
)
```




**Naive PnLs # (#naive-pnls)**

```
# Directional PnL

dix = dict_gd

sigx = dix["sigx"]
targ = dix["targx"][1]
cidx = dix["cidx"]
start = dix["start"]

naive_pnl = msn.NaivePnL(
        dfx,
        ret=targ,
        sigs=sigx,
        cids=cidx,
        start=start,
        bms=["USD_GLDXR_NSA", "USD_EQXR_NSA"],
    )

for sig in sigx:
    naive_pnl.make_pnl(
        sig,
        sig_op="raw",
        sig_add=0,
        sig_mult=1,
        thresh=4,
        rebal_freq="monthly",
        vol_scale=None,
        rebal_slip=1,
        pnl_name=sig + "_PZN0",
    )

for sig in sigx:
```

```
    naive_pnl.make_pnl(
        sig,
        sig_op="raw",
        sig_add=2,
        sig_mult=1.2,
        thresh=2,
        rebal_freq="monthly",
        vol_scale=None,
        rebal_slip=1,
        pnl_name=sig + "_PZN1",
    )

naive_pnl.make_long_pnl(label="Long only", vol_scale=None, leverage=1)

dix["pnls"] = naive_pnl

dix = dict_gd

sigx = dix["sigx"]
cidx = dix["cidx"]
start = dix["start"]
naive_pnl = dix["pnls"]
pnls = [sig + "_PZN0" for sig in sigx[:1]] + ["Long only"]

pnl_labels = {
    'MACRO_SUPPORT_ZN_PZN0': "Long-short strategy based on macro support score, vol-targeted",
    'Long only': "Long only position, vol-targeted",
}

naive_pnl.plot_pnls(
    pnl_cats=pnls,
    pnl_cids=["ALL"],
    start=start,
    title="Gold future PnLs: Long-short strategy versus long only positions",
    title_fontsize=16,
    figsize=(13, 7),
    compounding=False,
    xcat_labels=pnl_labels,
)

display(naive_pnl.evaluate_pnls(pnl_cats=pnls))

naive_pnl.signal_heatmap(
    pnl_name=sigx[0] + "_PZN0",
    title="Macro strategy signals for gold long-short strategy",
    title_fontsize=16,
    freq="q",
    start=start,
    figsize=(16, 1),
)
```

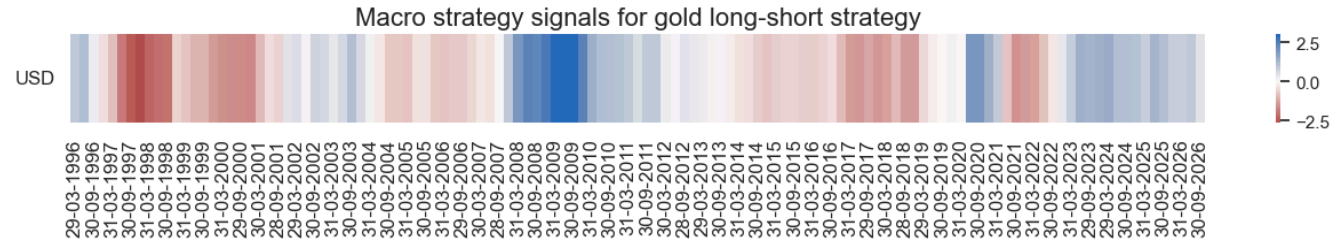



| xcat | MACRO_SUPPORT_ZN_PZN0 | Long only |
|---|---|---|
| Return % | 6.57432 | 4.056666 |
| St. Dev. % | 14.083415 | 11.888141 |
| Sharpe Ratio | 0.466813 | 0.341236 |
| Sortino Ratio | 0.66551 | 0.484902 |
| Max 21-Day Draw % | -23.0403 | -15.552025 |
| Max 6-Month Draw % | -30.290149 | -28.858088 |

| xcat | MACRO_SUPPORT_ZN_PZN0 | Long only |
|---|---|---|
| **Peak to Trough Draw %** | -3[illegible]2 | -7[illegible]8839 |
| **Top 5% Monthly PnL Share** | [illegible]994594 | 1.23[illegible]549 |
| **USD_GLDXR_NSA correl** | 0.1[illegible] | [illegible]916953 |
| **USD_EQXR_NSA correl** | 0.026067 | -0.010003 |
| **Sharpe Stability Ratio** | 2.75293 | 2.056395 |
| **Max Draw Recovery (months)** | 18.190476 | 144.47619 |
| **Prob. Sharpe Ratio > 0.25** | 0.863686 | 0.73343 |
| **Prob. Sharpe Ratio > 0.5** | 0.288125 | 0.208982 |
| **Prob. Sharpe Ratio > 0.75** | 0.013388 | 0.012444 |
| **Traded Months** | 368 | 369 |

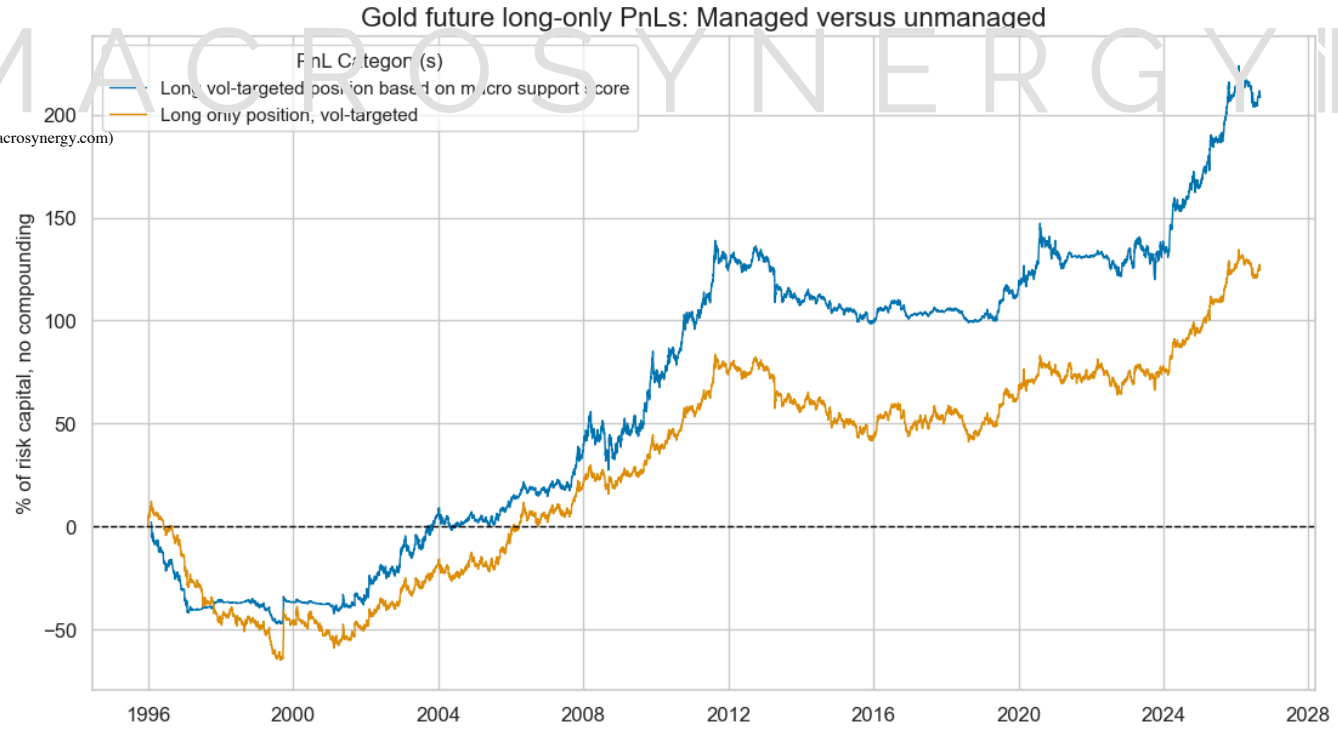




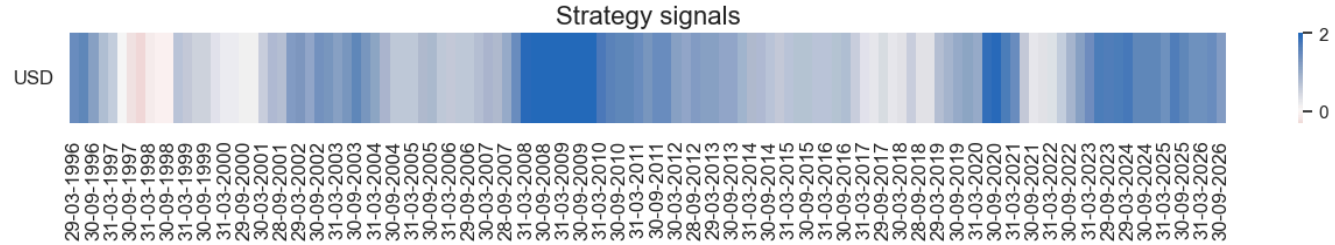



```
dix = dict_gd

sigx = dix["sigx"]
cidx = dix["cidx"]
start = dix["start"]
naive_pnl = dix["pnls"]
pnls = [sig + "_PZN1" for sig in sigx[:1]] + ["Long only"]

pnl_labels = {
    'MACRO_SUPPORT_ZN_PZN1': "Long vol-targeted position based on macro support score",
    'Long only': "Long only position, vol-targeted",
}

naive_pnl.plot_pnls(
    pnl_cats=pnls,
    pnl_cids=["ALL"],
    start=start,
    title="Gold future long-only PnLs: Managed versus unmanaged",
    title_fontsize=16,
    figsize=(13, 7),
    compounding=False,
    xcat_labels=pnl_labels,
)

display(naive_pnl.evaluate_pnls(pnl_cats=pnls))

naive_pnl.signal_heatmap(
    pnl_name=sigx[0] + "_PZN1",
    title="Strategy signals",
    title_fontsize=16,
    freq="q",
    start=start,
    figsize=(16, 1),
)
```

(https://macrosynergy.com)




| xcat | MACRO_SUPPORT_ZN_PZN1 | Long only |
|---|---|---|
| Return % | 6.815445 | 4.056666 |
| St. Dev. % | 13.226768 | 11.888141 |
| Sharpe Ratio | 0.515277 | 0.341236 |
| Sortino Ratio | 0.726021 | 0.484902 |
| Max 21-Day Draw % | -19.068316 | -15.552025 |
| Max 6-Month Draw % | -26.493228 | -28.858088 |
| Peak to Trough Draw % | -49.396278 | -77.158839 |
| Top 5% Monthly PnL Share | 0.934465 | 1.234549 |
| USD_GLDXR_NSA correl | 0.864415 | 0.916953 |
| USD_EQXR_NSA correl | 0.004102 | -0.010003 |
| Sharpe Stability Ratio | 2.613739 | 2.056395 |
| Max Draw Recovery (months) | 96.666667 | 144.47619 |
| Prob. Sharpe Ratio > 0.25 | 0.852153 | 0.73343 |
| Prob. Sharpe Ratio > 0.5 | 0.300726 | 0.208982 |
| Prob. Sharpe Ratio > 0.75 | 0.018294 | 0.012444 |
| Traded Months | 368 | 369 |



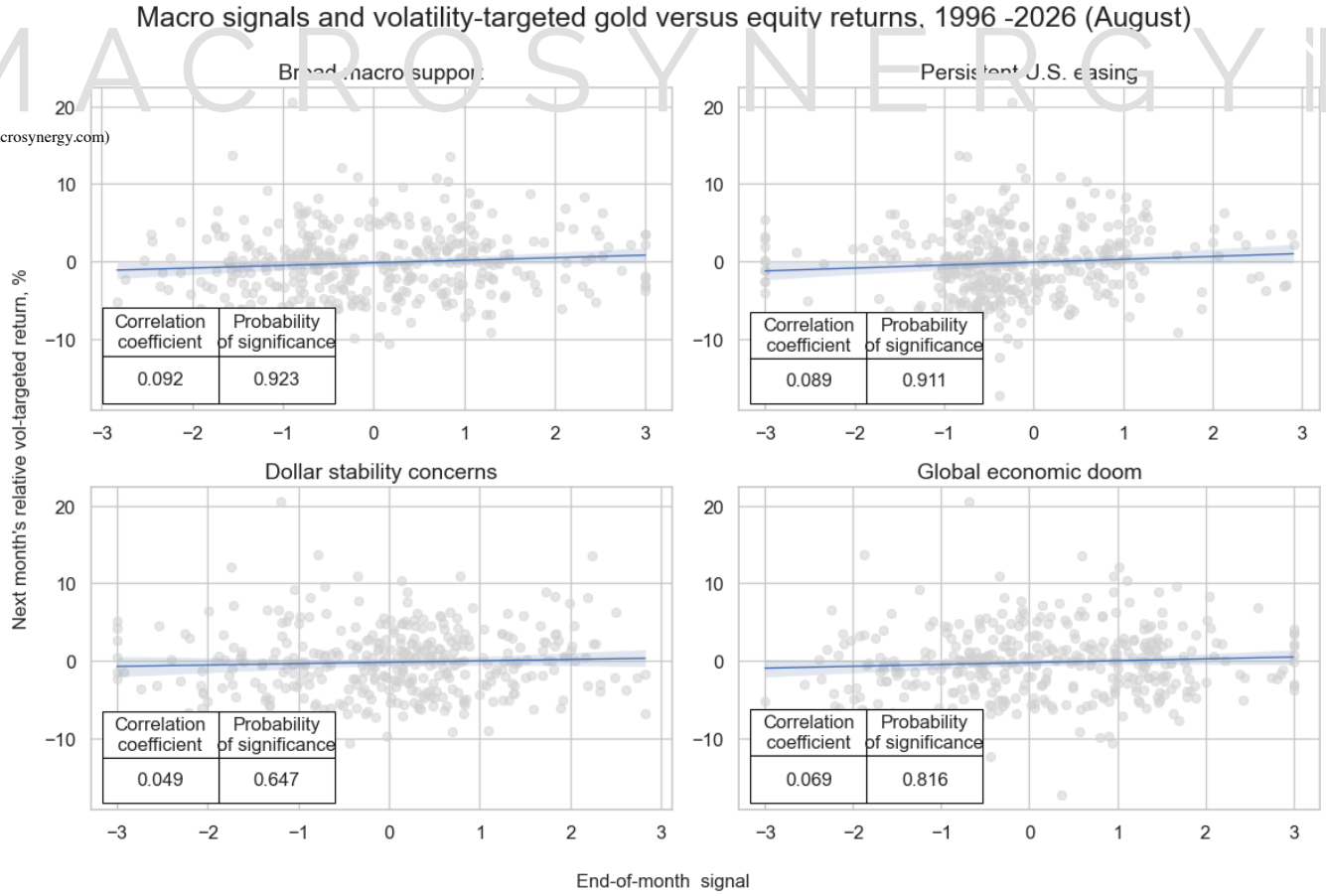

## Gold versus equity # (#gold-versus-equity)

### Specs and regression test # (#id1)

```
# Relative specs

dict_re = {
    "sigx": macro_factors_zn,
    "targx": ['GLDvEQXR_VT10'],
    "cidx": ["USD"],
    "start": "1996-01-01",
    "crs": None,
    "srr": None,
    "pnls": None,
}

# Relative test regressions
```

```
dix = dict_re

sigs = dix["sigx"]
targs = dix["targx"]
cidx = dix["cidx"]
start = dix["start"]

# Initialize the dictionary to store CategoryRelations instances

dict_cr = {}

for targ in targs:
    for sig in sigs:
        lab = sig + "_" + targ
        dict_cr[lab] = msp.CategoryRelations(
            dfx,
            xcats=[sig, targ],
            cids=cidx,
            freq="M",
            lag=1,
            xcat_aggs=["last", "sum"],
            start=start,
        )

dix["crs"] = dict_cr
```

```
# Relative plots

dix = dict_re
dict_cr = dix["crs"]
sigs = dix["sigx"]
targs = dix["targx"]

crs = list(dict_cr.values())
crs_keys = list(dict_cr.keys())
ncol = 2
nrow = 2

scatter_labels = {
    'MACRO_SUPPORT_ZN_GLDvEQXR_VT10': "Broad macro support",
    'FINMON_EASE_ZN_GLDvEQXR_VT10': "Persistent U.S. easing",
    'DOLLAR_STAB_ZN_GLDvEQXR_VT10': "Dollar stability concerns",
    'GLOBAL_DOOM_ZN_GLDvEQXR_VT10': "Global economic doom",
}

msv.multiple_reg_scatter(
    cat_rels=crs,
    ncol=ncol,
    nrow=nrow,
    figsize=(12, 8),
    prob_est="pool",
    coef_box="lower left",
    title="Macro signals and volatility-targeted gold versus equity returns, 1996 –2026 (August)",
    title_fontsize=18,
    subplot_titles=[scatter_labels[cr] for cr in crs_keys],
    share_axes=False,
    xlab="End-of-month  signal",
    ylab="Next month's relative vol-targeted return, %",
    coef_box_font_size=12,
)
```

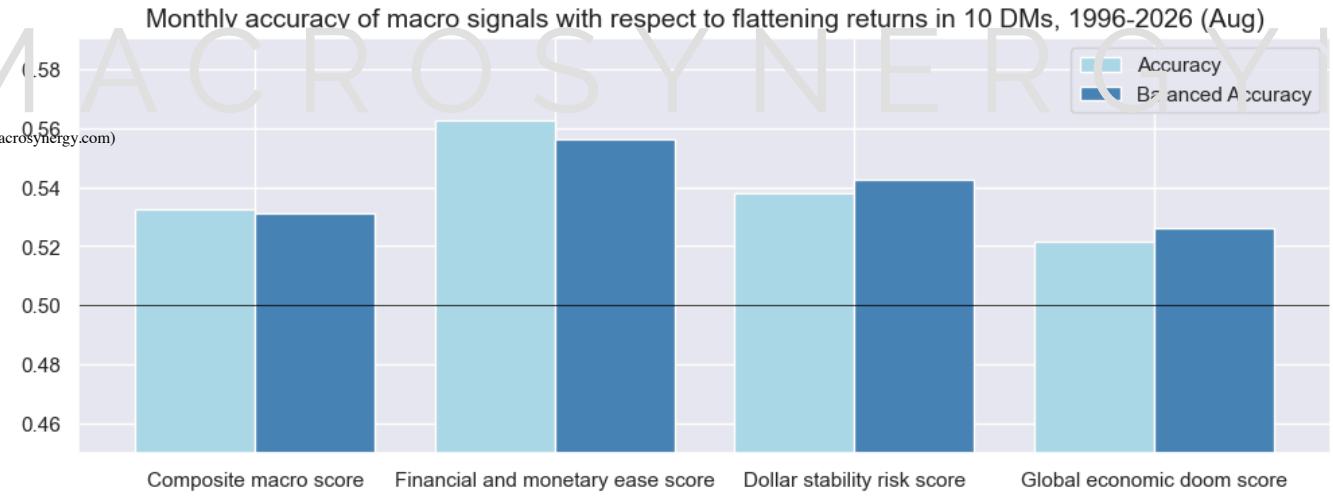



**Accuracy and correlation check # (#id2)**

```
# Relative accuracy

dix = dict_re

sigs = dix["sigx"]
targs = dix["targx"]
cidx = dix["cidx"]
start = dix["start"]

srr = mss.SignalReturnRelations(
    dfx,
    cids=cidx,
    sigs=sigs,
    rets=targs,
    freqs="M",
    start=start,
)

display(srr.multiple_relations_table().round(3))

dix["srr"] = srr
```

| | | | | accuracy | bal_accuracy | pos_sigr | pos_retr | pos_prec | neg_prec | pearson | pearson_pval | kendall | kendall_pval | auc |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Return | Signal | Frequency | Aggregation | | | | | | | | | | | |
| GLDvEQXR_VT10 | DOLLAR_STAB_ZN | M | last | 0.538 | 0.543 | 0.554 | 0.462 | 0.500 | 0.585 | 0.049 | 0.353 | 0.047 | 0.182 | 0.542 |
| | FINMON_EASE_ZN | M | last | 0.562 | 0.556 | 0.351 | 0.462 | 0.535 | 0.577 | 0.089 | 0.089 | 0.070 | 0.046 | 0.551 |
| | GLOBAL_DOOM_ZN | M | last | 0.522 | 0.526 | 0.554 | 0.462 | 0.485 | 0.567 | 0.069 | 0.184 | 0.052 | 0.133 | 0.526 |
| | MACRO_SUPPORT_ZN | M | last | 0.533 | 0.531 | 0.478 | 0.462 | 0.494 | 0.568 | 0.092 | 0.077 | 0.066 | 0.060 | 0.531 |

```
# Relative accuracy plots

dix = dict_re
srr = dix["srr"]
targ = dix["targx"][0]


srr.accuracy_bars(
    view="signals",
    ret=targ,
    size=(12, 4),
    title="Monthly accuracy of macro signals with respect to flattening returns in 10 DMs, 1996-2026 (Aug)",
    title_fontsize=14,
    x_labels=sig_labels,
    x_label_rotation=45,
)
```

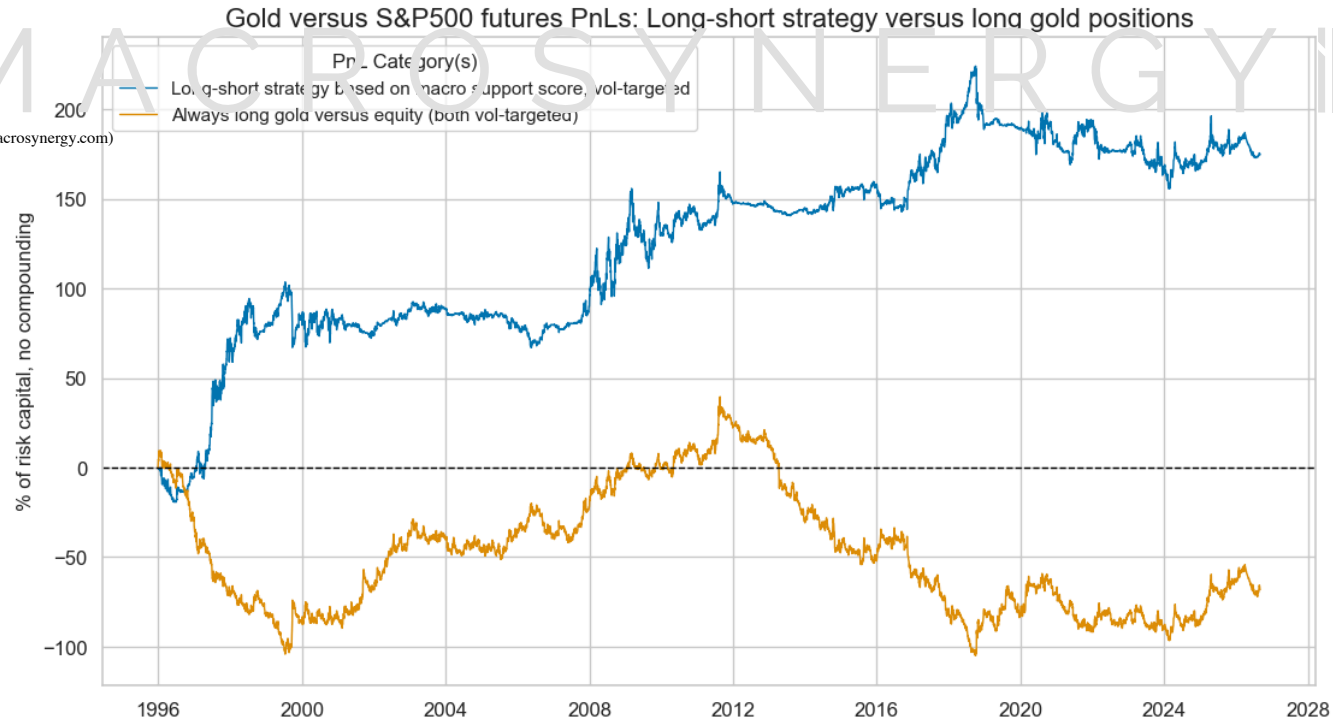




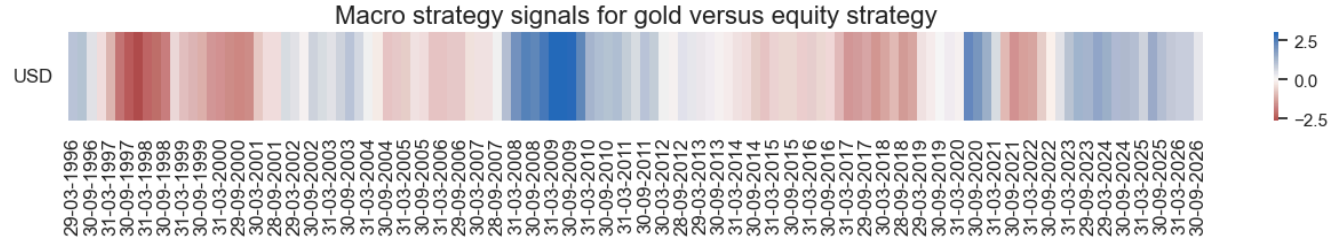

**Naive PnLs # (#id3)**

```
# Relative PnL

dix = dict_re

sigx = dix["sigx"]
targ = dix["targx"][0]
cidx = dix["cidx"]
start = dix["start"]



naive_pnl = msn.NaivePnL(
        dfx,
        ret=targ,
        sigs=sigx,
        cids=cidx,
        start=start,
        bms=["USD_GLDXR_NSA", "USD_EQXR_NSA"],
    )

for sig in sigx:
    naive_pnl.make_pnl(
        sig,
        sig_op="raw",
        sig_add=0,
        thresh=4,
        rebal_freq="weekly",
        vol_scale=None,
        rebal_slip=1,
        pnl_name=sig + "_PZN",
    )

naive_pnl.make_long_pnl(label="Long gold", vol_scale=None)

dix["pnls"] = naive_pnl

dix = dict_re

sigx = dix["sigx"]
cidx = dix["cidx"]
start = dix["start"]
naive_pnl = dix["pnls"]
pnls = [sig + "_PZN" for sig in sigx[:1]] + ["Long gold"]

pnl_labels = {
    'MACRO_SUPPORT_ZN_PZN': "Long-short strategy based on macro support score, vol-targeted",
    'Long gold': "Always long gold versus equity (both vol-targeted)",
}

naive_pnl.plot_pnls(
    pnl_cats=pnls,
    pnl_cids=["ALL"],
    start=start,
    title="Gold versus S&P500 futures PnLs: Long-short strategy versus long gold positions",
    title_fontsize=16,
    figsize=(13, 7),
    compounding=False,
    xcat_labels=pnl_labels,
)

display(naive_pnl.evaluate_pnls(pnl_cats=pnls))

naive_pnl.signal_heatmap(
    pnl_name=sigx[0] + "_PZN",
    title="Macro strategy signals for gold versus equity strategy",
    title_fontsize=16,
    freq="q",
    start=start,
    figsize=(16, 1),
)
```

(https://macrosynergy.com)




| xcat | MACRO_SUPPORT_ZN_PZN | Long gold |
|---|---|---|
| **Return %** | 5.707007 | -2.220118 |
| **St. Dev. %** | 20.601441 | 16.967425 |
| **Sharpe Ratio** | 0.27702 | -0.130846 |
| **Sortino Ratio** | 0.390061 | -0.190391 |
| **Max 21-Day Draw %** | -30.33564 | -18.001757 |
| **Max 6-Month Draw %** | -41.650885 | -43.828865 |
| **Peak to Trough Draw %** | -68.584246 | -144.552983 |
| **Top 5% Monthly PnL Share** | 1.408772 | -2.785995 |
| **USD_GLDXR_NSA correl** | 0.077054 | 0.639986 |
| **USD_EQXR_NSA correl** | -0.063288 | -0.621115 |
| **Sharpe Stability Ratio** | 1.660167 | -0.813792 |
| **Max Draw Recovery (months)** | 98.380952 | 186.714286 |
| **Prob. Sharpe Ratio > 0.25** | 0.322539 | 0.009826 |
| **Prob. Sharpe Ratio > 0.5** | 0.00492 | 0.000059 |
| **Prob. Sharpe Ratio > 0.75** | 0.000001 | 0.0 |
| **Traded Months** | 369 | 369 |




Contents

By Macrosynergy Ltd.

time macroeconomic trends in developed and emerging countries, transforming them into macro systematic quantamental investment strategies. In June 2020 Macrosynergy and J.P. Morgan started a collaboration to scale the quantamental system and to popularize tradable economics across financial markets.



FOLLOW US

(https://macrosynergy.com)

SIGN UP



Manage consent